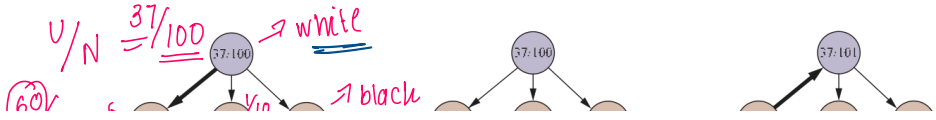
Monte Carlo Tree Search

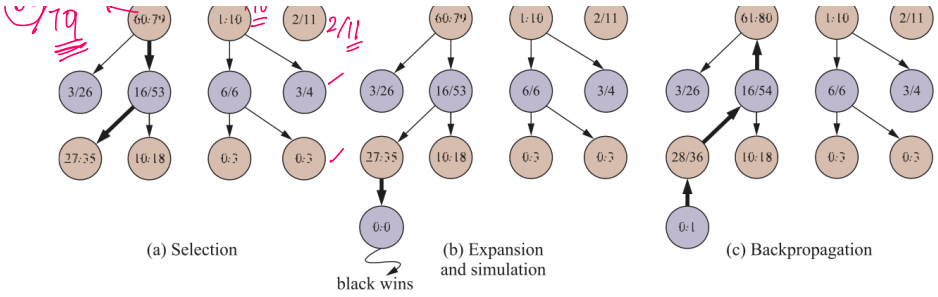
Friday, 24 October 2025 20:40

- basic MCTS strategy does not use a heuristic evaluation function
- Value of a state is estimated as the average utility over a number of simulations of complete games starting from the state
- Simulation/playout/rollout chooses moves first for one player, than for the other, repeating until a terminal position is reached
- Rules of game determine who wins and loses
- To get useful information from the playout we need a playout policy that biases the moves towards good ones.
  - from what positions do we start the playouts
  - how many playouts do we allocate to each position?
- Monte Carlo search, is to do N simulations starting from the current state of the game, and track which of the possible moves from the current position has the highest win percentage.
- This may or may not lead to optimal play
- we need a selection policy that selectively focuses the computational resources on the important parts of the game tree
  - Exploration - states that have had few playouts
  - Exploitation - states that have done well in past

This helps get a more accurate estimate of their value

- **Selection**: Starting at the root of the search tree, we choose a move (guided by the selection policy), leading to a successor node, and repeat that process, moving down the tree to a leaf. Figure 5.10(a) shows a search tree with the root representing a state where white has just moved, and white has won 37 out of the 100 playouts done so far. The thick arrow shows the selection of a move by black that leads to a node where black has won 60/79 playouts. This is the best win percentage among the three moves, so selecting it is an example of exploitation. But it would also have been reasonable to select the 2/11 node for the sake of exploration—with only 11 playouts, the node still has high uncertainty in its valuation, and might end up being best if we gain more information about it. Selection continues on to the leaf node marked 27/35.
- **Expansion**: We grow the search tree by generating a new child of the selected node; Figure 5.10(b) shows the new node marked with 0/0. (Some versions generate more than one child in this step.)
- **Simulation**: We perform a playout from the newly generated child node, choosing moves for both players according to the playout policy. These moves are *not* recorded in the search tree. In the figure, the simulation results in a win for black.
- **Back-propagation**: We now use the result of the simulation to update all the search tree nodes going up to the root. Since black won the playout, black nodes are incremented in both the number of wins and the number of playouts, so 27/35 becomes 28/26 and 60/79 becomes 61/80. Since white lost, the white nodes are incremented in the number of playouts only, so 16/53 becomes 16/54 and the root 37/100 becomes 37/101.



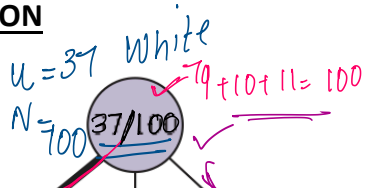


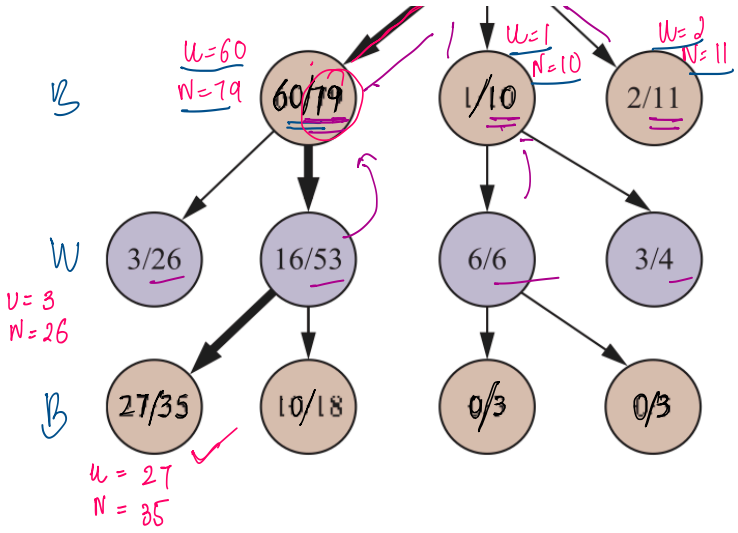
- repeat these four steps either for a set number of iterations, or until the allotted time has expired, and then return the move with the highest number of playouts.
- Selection policy: *upper confidence bounds applied to trees*

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\log N(\text{PARENT}(n))}{N(n)}}$$

  - U(n)is the total utility of all playouts that went through node n
  - N(n)is the number of playouts through node n
  - PARENT(n) is the parent node of n in the tree
  - U(n)/N(n) is the exploitation term: the average utility of n. term with the square root is the exploration term: it has the count N(n) in the denominator, which means the term will be high for nodes that have only been explored a few times. In the numerator it has the log of the number of times we have explored the parent of n. *This means that if we are selecting n some non-zero percentage of the time, the exploration term goes to zero as the counts increase, and eventually the playouts are given to the node with highest average utility.*
  - C is a constant that balances exploitation and exploration.There is a theoretical argument that C should be √2.
- When the iterations terminate, the move with the highest number of playouts is returned. You might think that it would be better to return the node with the highest average utility, but the idea is that a node with 65/100 wins is better than one with 2/3 wins, because the latter has a lot of uncertainty. In any event, the UCB1 formula ensures that the node with the most playouts is almost always the node with the highest win percentage, because the selection process favors win percentage more and more as the number of playouts goes up.

PRACTICE QUESTION





(a) Selection

Selection

Left child (60/79)

$MCTS = \frac{60}{79} + 1.4\sqrt{\frac{\ln 100}{79}}$

$= 1.0975$

Middle child (1/10)

$MCTS = \frac{1}{10} + 1.4\sqrt{\frac{\ln 100}{10}}$

$= 1.05$

Right child (2/11)

$MCTS = \frac{2}{11} + 1.4\sqrt{\frac{\ln 100}{11}}$

$= 1.087$

2nd layer

Left child (3/26)

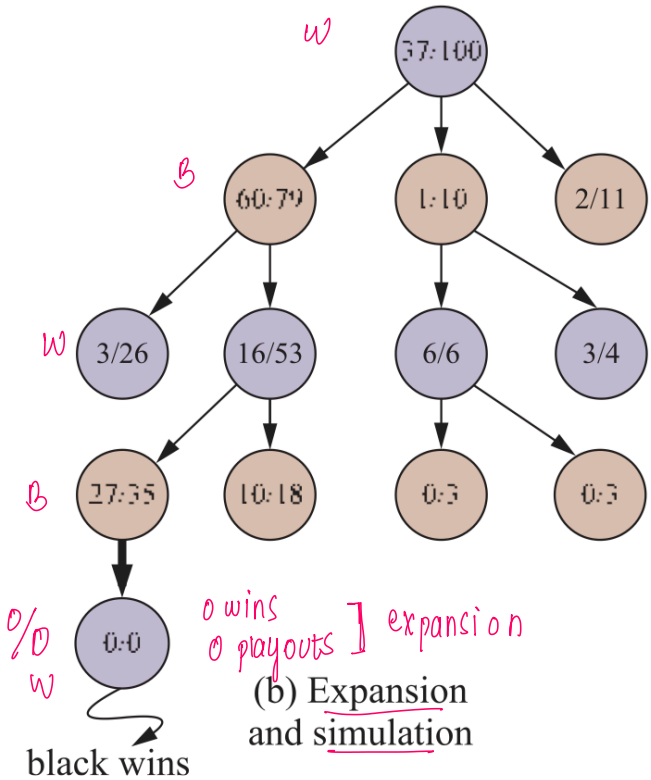
$= \frac{3}{26} + 1.4\sqrt{\frac{\ln 79}{26}}$

$= 0.689$

Right child (16/53)

$= \frac{16}{53} + 1.4\sqrt{\frac{\ln 79}{53}}$

$= 0.7089$



(b) Expansion and simulation

0 wins
0 playouts
] expansion

27/35 gets new child

There are no simulation hence 0/0.

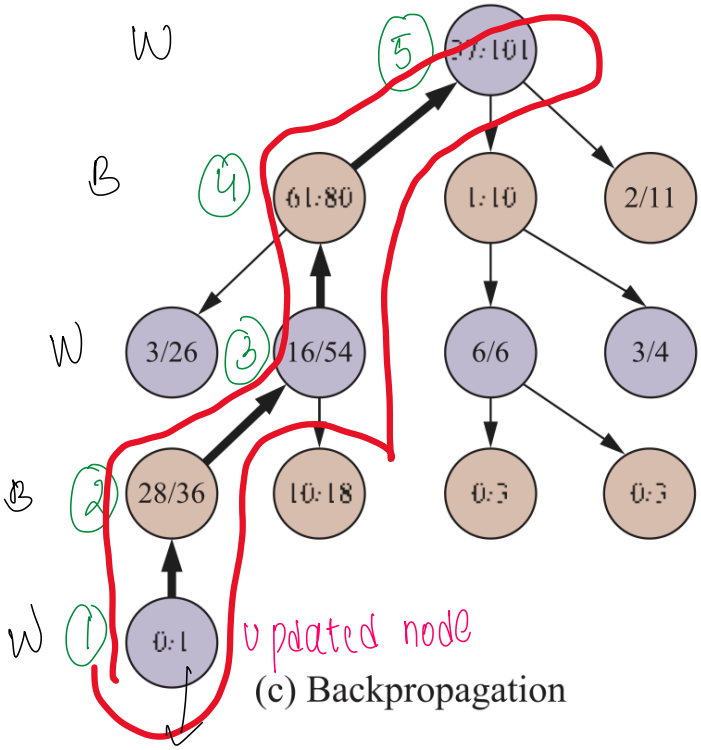
1st simulation is run:

Black wins

Child becomes 0/1

1 simulation

0 wins (opponent won)



(c) Backpropagation

(2) since black won
27/35 became 28/36
simulation = 35+1 = 36 ✓
wins for black = 27+1 = 28

(3) 16/53 becomes 16/54
simulation = 53+1 = 54
wins = 16+0 = 16

(4) 60/79 becomes 61/80
simulation = 79+1 = 80
wins for black = 60+1 = 61

(5) 37/100 becomes 37/101
simulation = 100+1 = 101
wins = 37+0 = 37 ✓

- **UCB1** balances *exploitation* (average win rate) and *exploration* (uncertainty term).
- **Expansion** adds a new leaf node when an unvisited action is encountered. 0/0
- **Simulation** provides a random outcome for that leaf.
- **Backpropagation** updates statistics up the path.
- The **most visited** child at the root is ultimately selected as the best move.